Conversation Export: AI Credit Analysis App Build

Exported November 8, 2026. Upload this file to a new chat and say: "Continue from this conversation. Next step: I will share my credit spread model."

USER

Would it be a good wffective idea to develop an ai automated private credit spread analysis, and public equity credit and financial analysis tool that automatically takes in the financial reporting data available and conducts a thorough credit analysis by taking in qualitative public info into a portion of the analysis, analyzing macro economic data thats available to analyze the companys economic climate exposure, raise alert to potential accounting errors in the financials, and provide a thorough financial analysis of the company. You could either type in the ticker to the app to then generate the output from the most recent publicly available info, you could then have toggle settings on if you want a quarterly or annual report within a certain time range or both, and you could also have a section that stress test the company to determine how much bandwidth it has before it goes into a distressed situation. Is this something doable? The credit analysis would havr a credit rating output based on the similar weights Standard and Poors and Moodys applies and it would have a separate strictly technical financial credit rating that doesnt account for the macro economic and trends going on in the market, along with the volatility of the overall swing in financial performance

CLAUDE

Yes, it's doable and a good idea, with caveats on where the real difficulty and competition sit.

What's very feasible

- Pulling financials from SEC EDGAR (free XBRL data) or paid providers (FactSet, S&P Capital IQ, Polygon, Financial Modeling Prep)
- Computing standard credit metrics: leverage, interest coverage, FCF/debt, liquidity, working capital trends
- Quarterly/annual toggles and date ranges
- Stress tests: shock revenue, margins, and rates, then show how far EBITDA can fall before covenants or coverage break (a reverse stress test is the most useful version)
- LLM summarization of 10-K/10-Q text, MD&A, risk factors, and earnings call transcripts
- Macro overlay using FRED data (rates, GDP, credit spreads, sector indicators)
- Two separate scores: a "pure financial" rating and a "full" rating adding macro, trend, and volatility

What's harder

- **Accounting error detection.** Rule-based checks work well (Beneish M-Score, Altman Z, Piotroski F, accruals ratios, balance sheet tie-outs, receivables vs. revenue divergence). Catching real fraud is much harder, so frame outputs as "flags for review," not conclusions.
- **Replicating S&P/Moody's weights.** Their methodologies are published by sector (e.g., Moody's scorecards weight scale, business profile, leverage, coverage, financial policy), but much of the final rating is analyst judgment. You can build a *scorecard-based implied rating* calibrated against actual ratings, but call it "implied" and don't present it as an agency rating.
- **Data quality.** XBRL tagging is inconsistent across companies, and normalizing it (adjusting for leases, one-time items, non-GAAP) is where most of the engineering effort goes.

- **LLM reliability.** Numbers should come from deterministic code, not the model. Use the LLM only for qualitative extraction, with citations to source filings, and have it explain results rather than calculate them.
- **Private credit.** This is the weak spot. Private companies don't file publicly, so you'd need users to upload financials (PDFs, Excel, compliance certificates), which is a different product from the ticker-lookup flow. Spread analysis there is mostly a document-extraction problem.

Suggested architecture

1. Data layer: ingest and normalize filings into a clean standard schema
2. Calculation engine: deterministic ratios, scores, and stress models
3. Rating model: scorecard with transparent weights, calibrated against a dataset of agency ratings by sector
4. LLM layer: qualitative analysis, flagging, narrative writing, all with source citations
5. Output: report with the two ratings, drivers, alerts, and stress test

Is it worth building?

- *Competition:* Capital IQ, Bloomberg, Moody's CreditLens, and Fitch Solutions already do parts of this, and there are many AI startups on filings analysis. Your edge would be price, speed, and the integrated one-click output, probably for smaller funds, credit analysts at community banks, and independent investors.
- *Validation:* Backtest against actual rating changes and defaults. If your score doesn't anticipate downgrades, it's not valuable.
- *Regulatory:* If you present ratings commercially, be careful about positioning. Calling them "credit ratings" can touch NRSRO issues. "Implied credit score" is safer.

Practical path: Start with an MVP for public companies only, with financial ratios, Altman/Beneish flags, a reverse stress test, and an LLM-written summary. Test it on 50 companies with known ratings, then add the macro layer and the second rating, and only then consider private credit uploads.

Want me to sketch the scorecard structure or the data schema first?

USER

I would provide an existing credit spread model so you could integrate that engineering. The toggle buttons on the app would enable you to select private credit analysis and upload the financials that you have available to then allow the system to conduct the analysis. I would want various app layers that allow the user to drill into the various lenses they are seeking to understand and have it be clean, understanding and thorough with its reporting. Leases (financial and operating machinery leases) would be accounted for in long term or short term debt because it is a form of a debt payment instrument. There would also be an off balance sheet integration piece that allows a user to see how AI analyzes the off balance sheet items and see that specific lens in how it affects the companys position amd financial health

CLAUDE

Your additions make the product coherent: a mode toggle (public ticker vs. private upload), your own spread model as the calculation core, and drill-down layers. Here is how I'd shape each piece.

App layers (drill-down structure)

1. **Summary:** both ratings, key drivers, top alerts, distance-to-distress

2. **Financial profile:** normalized statements, ratios, trends, volatility
3. **Debt and leases:** adjusted debt build, maturity ladder, covenant headroom
4. **Off-balance-sheet lens:** every item found, its estimated debt-like value, and before/after metrics
5. **Accounting flags:** rule-based checks with severity and source references
6. **Macro and qualitative:** sector exposure, rate sensitivity, filing-text findings
7. **Stress test:** shocks and reverse stress
8. **Source trace:** every number and claim links to a filing page or uploaded document

Leases as debt

Treating them as debt is the right call, but handle the accounting differences:

- Under US GAAP (ASC 842), finance lease liabilities already sit in debt. Operating lease liabilities are on the balance sheet now but classified separately, so you add them to adjusted debt.
- Under IFRS 16, nearly all leases are on balance sheet already.
- Adjust EBITDA consistently: if operating lease liabilities go into debt, add back operating lease cost (or the interest-equivalent portion) so leverage isn't double-penalized. Show both "reported" and "lease-adjusted" metrics side by side.
- Split into current and long-term portions for the maturity ladder.
- For private companies, leases often sit only in footnotes or aren't recorded at all (small private firms may not follow ASC 842 rigorously), so the extractor needs a fallback: capitalize rent expense using a multiple (the old rating-agency approach was roughly 8x annual rent) and label it as an estimate.

Off-balance-sheet lens

This is your most differentiated feature, so make the AI's reasoning visible. Categories to detect:

- Purchase obligations and take-or-pay contracts
- Guarantees and letters of credit
- Unconsolidated VIEs and joint-venture debt
- Factoring, receivables securitization, supply chain finance
- Pension and OPEB underfunding
- Contingent liabilities and litigation reserves
- Earnouts, put options, and unfunded commitments

For each item, show: the source text excerpt, how the AI classified it, the probability-weighted debt-equivalent amount, and the effect on leverage, coverage, and the rating. A simple toggle ("include off-balance-sheet adjustments") that re-runs the whole model makes the impact tangible. Keep the amounts rule-based where possible (e.g., guarantees at a stated percentage of exposure) and let the AI do the detection and classification, with a confidence score and a "needs human review" flag.

Private credit specifics

- Uploads will be messy (PDFs, Excel, compliance certificates, management accounts), so build an extraction step with a review screen where the user confirms mapped line items before the model runs. This is the single most important trust feature.
- Support covenant definitions, since private deals use custom EBITDA add-backs. Let the user input the credit agreement terms so headroom is calculated against the actual covenants.
- Skip the macro/public-text sections when there's no public info, or let the user attach a peer set or sector for the macro overlay.

- Note the "implied rating" should be calibrated differently for private borrowers, which are usually smaller and more volatile than rated issuers.

Integrating your spread model

The key questions are what it takes as inputs (normalized financials, ratings, sector, maturity?) and what it outputs (spread estimate, relative value signal?). If you can share its structure or inputs and outputs, I can map exactly where it plugs into the calculation engine and what fields the data schema must supply.

Build order I'd suggest

Public ticker flow with core metrics and lease adjustment first, then your spread model, then the off-balance-sheet lens, then the stress test, then private uploads with the review screen.

Would you like me to draft the data schema or the off-balance-sheet detection logic first?

USER

So this is something you can build and we can work on together to have a great app buildout that is usable in real life?

CLAUDE

Yes, I can be a real build partner on this, and a usable app is a realistic goal.

What I can do

- Write the actual code: data ingestion, the normalization layer, the calculation engine, the scorecard, stress tests, the off-balance-sheet detection prompts, and the UI
- Integrate your spread model once I see how it works
- Build working prototypes here, such as a clickable version of the drill-down layers
- Design the schema, test cases, and validation approach
- Debug and iterate with you as you test

What I can't do alone

- Host or deploy it. You'd need a dev environment and cloud setup (or a developer to help with that).
- Pull live filings from this chat, since my sandbox can't reach SEC EDGAR. We'd build against sample filings you provide, then you'd run it against live data on your side.
- Validate it. You'd need to check outputs against real agency ratings and known defaults, which takes your domain judgment.
- Handle legal and compliance questions like rating positioning and data licensing. That needs a lawyer.

How I'd run it

1. You share your spread model (inputs, outputs, or the code or spreadsheet itself)
2. We lock the data schema and the lease-adjusted metrics
3. I build the public-ticker MVP in small, testable pieces, and you test each one on companies you know well
4. We add the off-balance-sheet lens, then the stress test, then private uploads

Starting with your spread model is the best first step, because it determines the schema. Can you share it, or describe its inputs and outputs?

STATE OF PLAY (for the next session)

Project: AI-driven credit analysis app covering public companies (ticker lookup) and private credit (financial uploads).

Agreed features: quarterly/annual/both toggle with date ranges; two ratings (full implied rating with macro, trend, and volatility, plus a strictly financial rating); accounting-error flags; stress test with reverse stress; lease-adjusted debt (finance and operating leases); off-balance-sheet lens; eight drill-down layers; source trace on every number.

Open item: user will supply their existing credit spread model (inputs, outputs, or code/spreadsheet) so it can be integrated into the calculation engine.

Next steps: (1) review the spread model, (2) lock the data schema and lease-adjusted metrics, (3) build the public-ticker MVP in testable pieces, (4) add off-balance-sheet lens, stress test, then private uploads with a review screen.

Constraints noted: sandbox cannot reach SEC EDGAR, so build against sample filings; hosting, validation against agency ratings, and legal/NRSRO positioning are the user's side.